

Verifier-Grounded Evaluation of LLM-Generated Network Configurations: A Preregistered NetConfig-Semantic Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a fully preregistered, artifact-anchored paired experiment (CAND-2026-001-NetConfig-Semantic-v1; preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37) comparing a deterministic template baseline to a single-pass LLM generator (declared_model_version = gpt-5-mini) on N = 120 synthetic intent→configuration tasks using a deterministic validator. Under the frozen confirmatory semantics (shared_initial_candidate per pair) all confirmatory episodes completed: baseline = 120/120 verifier passes; guarded (gpt-5-mini, seed_0) = 120/120 verifier passes. Paired difference = 0.00; 95% bootstrap percentile CI [0.00, 0.00] (10,000 resamples, seed = 1729); exact McNemar two-sided p = 1.0. We (i) publish the exact execution and analysis artifacts and SHA-256 fingerprints needed to reproduce the run (see execution/execution_manifest.json in the artifact bundle), (ii) diagnose—using archived artifacts—why the paired outcome is uninformative about independent generative reliability (preregistered shared_initial_candidate design, template-tight task synthesis, and validator–task coupling), and (iii) provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory design, validator diversity, and expanded task heterogeneity) that preserve reproducibility while producing informative independent comparisons. The immutable initial master prompt is archived (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdff1582bb39d15ba1).

I. INTRODUCTION

Motivation. Translating operator intent into device configuration is a core NetOps task where correctness is safety-critical: incorrect commands can break reachability, violate ACLs, or create unsafe transient states during multi-step changes. Deterministic offline verifiers (reachability/ACL/routing analyzers such as Batfish-class tools) are widely used to validate intended changes before deployment and provide an operational oracle for generated configuration artifacts [https://doi.org/10.1145/3603269.3604866]. Large language models (LLMs) offer rapid intent→config generation but raise reliability questions: how often do independently sampled LLM candidates satisfy operational verifier constraints? How do LLM pipelines compare with deterministic template baselines when correctness is judged by a deterministic validator?

Research question and contribution. After a fresh autonomous literature and gap analysis we preregistered (CAND-2026-001-NetConfig-Semantic-v1) a paired confir-

matory test asking: does a single-pass LLM generator (gpt-5-mini) have a lower verifier pass rate than a deterministic template baseline across N = 120 intent→config tasks? Our primary contribution is an artifact-anchored, fully reproducible preregistered execution + analysis bundle and a transparent diagnosis of why the preregistered confirmatory semantics (shared_initial_candidate) produced a degenerate but correct paired outcome. We (1) publish the exact artifacts and SHA-256 fingerprints required for byte-for-byte reruns; (2) report confirmatory counts and preregistered statistical inferences grounded in the archived traces (baseline = 120/120, guarded = 120/120; paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar p = 1.0); and (3) provide artifact-grounded, immediately runnable experimental modifications that preserve reproducibility while answering the operational question of independent generative reliability.

II. RELATED WORK

Verifier-grounded NetOps and intent→config. Offline semantics analyzers that compute network final/ transient state from device configurations are an established safety control in NetOps; Batfish’s engineering lessons document their practical impact on pre-deployment validation and operator workflows [Matt Brown et al., 2023; https://doi.org/10.1145/3603269.3604866]. Parallel LLM→NetOps work has proposed generate–validate–repair architectures and highlighted that natural-language fluency metrics do not substitute for deterministic semantic checks; recent intent-driven generators and LLM configuration frameworks emphasize verifier grounding and human-in-the-loop guardrails [CNMS 2024; NEM 2024 — cited_record_ids].

Methodological gap this study addresses. Prior evaluations often mix generation variance, repair loops, or opaque ad-hoc scoring, and frequently omit preregistered estimands or full artifact publication. Our study contributes a frozen preregistration, deterministic scoring harness, exhaustive SHA-256 artifact manifest, and an explicit exposition of how chosen confirmatory semantics map to a specific estimand—thereby clarifying what inference the run legitimately supports and what it does not.

III. METHODOLOGY

Preregistration and frozen design. We preregistered CAND-2026-001-NetConfig-Semantic-v1 prior to observing any model outputs; the preregistration artifact is archived (SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdebb71a37). The preregistration specified: paired confirmatory design across $N = 120$ tasks (40 tasks per topology class: edge, datacenter, multi-AS), primary estimand paired_success_rate_difference_guarded_minus_baseline, confirmatory generation_semantics = "shared_initial_candidate", initial canonical seed_0 per pair, validator = deterministic_netops_validator_v5, and the exact analysis plan (paired_binary_analysis_v1) including bootstrap parameters (10,000 resamples, seed = 1729). The full execution manifest and per-file SHA-256s are published (execution/execution_manifest.json; see artifact manifest in submission bundle).

Task synthesis and templates. Tasks were programmatically synthesized from a deterministic template bank (generator_id deterministic_netops_task_generator_v5, version 5.0). Each task manifest entry encodes: initial_state, expected_final_state, an explicit required_sequence (the minimal change sequence that must be executed to satisfy intent safely), per-task workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups), allowed_touched_fields, and a difficulty annotation (changed_interface_count, transient_rule_count). Representative task manifest entries and their SHA-256s are archived (e.g., execution/task_manifest.jsonl; see task examples under artifact_examples). Templates intentionally encode operationally meaningful transient constraints (e.g., make-before-break switchover, atomic MTU changes) so that verifier checks carry operational semantics rather than purely syntactic checks.

Model, orchestration, and generation semantics. The declared LLM was gpt-5-mini accessed via the hosted adapter family hosted_netops_gvr_v1 (execution/model_configuration.json; SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). The preregistered confirmatory generation semantics produced a single canonical candidate per pair (shared_initial_candidate) and reused that identical candidate for both baseline and guarded scoring episodes; the canonical candidate files are archived at execution/responses/*-shared-initial.txt (representative SHA-256s: task-000001 shared initial SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6ccb8..., task-000003 SHA-256 f7f4db249ecbbce...).

Deterministic validator and scoring. Each candidate was scored by deterministic_netops_validator_v5 (validator_version 5.0). The validator parses commands, normalizes configuration sequences, enforces per-task workflow_policies, runs transient safety checks (e.g., minimum_active_in_groups during the change), and computes final_state reachability

and ACL invariants. The scoring rule used for the confirmatory test was full_intent_constraint_validation: a binary pass (VALID_CONFIGURATION = 1) requires that all required_commands and transient constraints are satisfied. Each scoring episode emits a detailed JSON trace archived at execution/scoring/*.json (representative: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...).

Execution accounting, retries, contamination heuristics. Planned_episode_count = 240 (120 pairs $\times$ 2 conditions). Completed_episode_count = 240; model_calls_used = 120 (shared_initial generation per pair counted once); failed_episode_count = 0. A preregistered retry policy allowed up to two automated retries for infrastructure failures; none were required. Contamination heuristics (exact-match and normalized n-gram overlap) were run as preregistered and recorded (analysis/contamination_summary.csv). All provider call traces and call_cache artifacts are included (execution/provider_calls/*.json, execution/call_cache/*) to permit external exposure analysis.

IV. RESULTS

Confirmatory outcomes (artifact-anchored). All preregistered confirmatory scoring episodes completed. The archived confirmatory counts are: baseline_success_count = 120/120; guarded_success_count = 120/120; complete_pair_count = 120. The paired contingency table is degenerate: $n_{11} = 120$, $n_{10} = 0$, $n_{01} = 0$, $n_{00} = 0$ (see analysis/paired_contingency_table.csv; SHA-256 9f25790c...). The preregistered paired estimator (mean of guarded_i - baseline_i) = 0.00. Bootstrap inference (10,000 resamples; seed = 1729) yields a 95% percentile CI [0.00, 0.00]; exact McNemar two-sided $p = 1.0$. All analysis artifacts (condition_summary.csv SHA-256 9c77c96f..., paired_difference_figure.svg SHA-256 ae5b8e0c...) are archived for byte-for-byte verification.

Why the contingency is degenerate (artifact diagnosis). Two preregistered, observable design choices (both visible in the artifact bundle) explain the degenerate result. First, confirmatory generation_semantics = "shared_initial_candidate" intentionally reuses a single canonical candidate per task for both the baseline and guarded scoring episodes; those canonical candidates are archived at execution/responses/task-000XXX-shared-initial.txt (representative SHA-256s: task-000001 8f38c8be..., task-000002 ae967f70..., task-000003 f7f4db24...). Second, the task templates encode explicit required_sequences and transient workflow_policies (e.g., must_be_down_for_fields, minimum_active_in_groups) that the canonical candidates satisfy. Because deterministic_netops_validator_v5 deterministically maps the identical canonical input to the same pass/fail outcome, the paired counts are necessarily identical (n_{11} for all pairs). The scoring traces make this chain of logic directly auditable: e.g., execution/scoring/task-000001-guarded.json (SHA-256 0d56c1ad...) shows parsed_command_count = 4, required_command_count = 4, violation_count = 0 and contains before/after final_state

dictionaries; the corresponding shared_initial candidate file execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8be...) matches the candidate text used for both conditions.

Validator trace evidence (representative examples). Per-episode JSON scorer outputs include validator fields used in our interpretation: normalized_configuration, parsed_command_count, required_command_count, validation_before.final_state and validation_after.final_state, violation_count, and scorer_id/version. Example artifacts (all archived): - execution/scoring/task-000001-guarded.json (SHA-256 0d56c1ad...): parsed_command_count=4; required_command_count=4; valid=true; final_state mapping present. Shared candidate: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8be...). - execution/scoring/task-000002-guarded.json (SHA-256 993b8eb3...); shared candidate: execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70...). - execution/scoring/task-000003-guarded.json (SHA-256 dad1cf4c...); shared candidate: execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db24...). These traces demonstrate that the validator confirmed required sequences and transient constraints for the canonical candidates; full JSON traces are available under execution/scoring/ for audit.

Provider-call and execution accounting evidence. The execution manifest records provider call traces and caching (execution/provider_calls/*.json and execution/call_cache/*). Key archived counts: planned_episode_count=240, completed_episode_count=240, model_calls_used=120, maximum_model_calls=240. No failed provider calls occurred; the preregistered retry policy permitted up to two retries for infrastructure failures but none were invoked. The exhaustive execution artifact manifest (execution/execution_manifest.json) contains per-file SHA-256 fingerprints for deterministic reproduction.

Contamination checks and caveats. The preregistered contamination heuristics (exact match and normalized n-gram overlap thresholds) produced no confirmatory flags (analysis/contamination_summary.csv is empty under the preregistered thresholds). As stated in the preregistration and Disclosure, the absence of a flag is not proof of zero pretraining exposure; full provider call traces and call_cache artifacts are archived to permit more intensive exposure analyses by third parties.

Implication of the numeric results. The confirmatory statistics are internally correct and properly reported for the preregistered estimand: they test whether a deterministic verifier treats identical artifacts differently across conditions. They do not estimate the operational quantity of independent LLM generative reliability (the probability that independently sampled LLM candidates satisfy verifier constraints). The archived artifacts both demonstrate why (shared canonical inputs) and provide all materials necessary to run the alternate, informative protocols (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification) without any nondeterministic gaps in reproducibility.

V. DISCUSSION

Expanded, artifact-grounded interpretation and actionable diagnostics.

Precise inferential scope. The confirmatory analysis correctly answers the preregistered estimand as written: under shared_initial_candidate semantics, does the validator produce different pass/fail outcomes between two labeled scoring episodes? The archived contingency (analysis/paired_contingency_table.csv SHA-256 9f25790c7eeafb3...) shows a logically inevitable answer of "no difference" because the input artifact was identical per pair. Readers must therefore interpret the reported estimator (paired difference = 0.00; bootstrap CI [0.00,0.00]; McNemar $p = 1.0$) as a validation of the execution/analysis pipeline's determinism under identical inputs—not as evidence that the LLM would produce verifier-valid outputs under independent sampling in realistic deployments.

Artifact evidence chain that produced degeneracy. Three artifact classes in the bundle make the causal chain auditable: (1) canonical candidate files (execution/responses/*-shared-initial.txt; e.g., task-000001 SHA-256 8f38c8becd7e1e36..., task-000002 SHA-256 ae967f70dfb6c...), (2) per-episode scoring traces that record parsing and validation outcomes (execution/scoring/*.json; e.g., task-000001-guarded.json SHA-256 0d56c1add7c5a57f...), and (3) the paired contingency and condition summaries (analysis/paired_contingency_table.csv SHA-256 9f25790c..., analysis/condition_summary.csv SHA-256 9c77c96f37c4b6ff...). These three artifact types form a deterministic provenance path: shared_initial candidate → deterministic validator → identical binary score. The bundle contains the exact SHA-256 for each artifact enabling byte-for-byte verification of this chain without re-running the model (execution/execution_manifest.json lists the full manifest).

Why this matters for operational evaluation. Operational decision makers care about independent generative reliability: the probability that an LLM sampled anew (or an LLM ensemble, or an agentic pipeline) will produce a verifier-valid change under realistic prompt/seed/temperature variability and natural-language intent diversity. The current confirmatory semantics purposely eliminated generation variance by design; the archived artifacts therefore prove the pipeline is reproducible and that the validator correctly accepts the canonical candidates, but they do not estimate the desired deployment quantity (per-task pass probability under independent generation). The distinction is subtle but consequential: reproducible deterministic scoring is necessary for trustworthy pipelines, but it is not sufficient to establish generative robustness.

Runnable, artifact-grounded experiment modifications (preserving reproducibility). Using only the supplied artifacts and configuration files one can deterministically produce informative reruns that estimate independent generative reliability without changing validator code or the template bank: (A) independent_per_condition confirmatory rerun — for each

pair generate two independent candidates (one per condition) using distinct seeded calls to the archived generator (the generator id and configuration are archived at `execution/model_configuration.json` SHA-256 `a40e96e2...`), then rescore producing a nondegenerate paired table; (B) multi-seed confirmatory design — for each condition draw $K \geq 3$ independent seeds per task and compute per-condition pass probabilities and their bootstrap CIs (this was included as a preregistered exploratory plan); (C) validator diversification — add a second verifier (for example, an independent reachability or ACL analyzer and report intersection and union pass rates) to reduce estimator sensitivity to any single validator’s interpretation of `workflow_policies`. Each of these modifications is fully implementable using only the published code, seeds, and provider-call traces: because `execution/provider_calls/*.json` and `execution/call_cache/*` are included, external auditors can replay, re-seed, and deterministically re-score without needing private provider access to cached model outputs (the `call_cache` entries themselves contain the exact request/response fingerprints used in the run).

Practical tradeoffs and design guidance. Independent_per_condition sampling increases statistical informativeness but raises computational cost and exposure-management complexity. Multi-seed designs improve precision at the cost of increased model calls; the preregistered plan bounded calls conservatively (planned `maximum_model_calls` = 240) and specified stopping/retry rules; any rerun should adopt analogous resource bounds. Validator diversification improves construct validity but requires operational alignment: teams must decide whether the union (lenient) or intersection (conservative) of validators is the relevant deployment criterion. Importantly, all of these decisions are traceable and auditable using the provided manifest and SHA-256 list; reproducible governance is feasible by design.

Threats to validity (artifact-identified, expanded). Internal validity: the `shared_initial` design intentionally removed generation variance, producing a correct but uninformative confirmatory outcome. Construct validity: deterministic task templates and a single validator that enforces template constraints can inflate pass rates for canonical candidates; archived validator traces show that `required_sequence` checks were satisfied in all confirmed passes (see `execution/scoring/task-000003-guarded.json` SHA-256 `dad1cf4cfd80b4cc...`). External validity: the experiment used a single declared model (gpt-5-mini) and synthetic intent templates; results do not generalize to different LLM families, open natural-language intents, or production topologies. Conclusion validity: the degenerate contingency yields trivial hypothesis test outputs; interpreting $p = 1.0$ as evidence of equal operational reliability would be a category error. All these threats are documented and evidenced in the artifact bundle, enabling third parties to quantify them by rerunning the alternative designs described above.

Operational implications and recommended forensic checklist. For practitioners evaluating LLM→NetOps pipelines we

offer an artifact-based checklist grounded in this study’s materials: (1) preregister the estimand and generation semantics (shared vs independent) and archive both; (2) publish canonical candidate files, provider call traces, and validator traces with SHA-256s to enable third-party audit; (3) when estimating generative robustness, use independent_per_condition sampling and multi-seed replication; (4) require validator diversification for high-stakes rollouts; (5) perform contamination/exposure checks over published candidates and provider traces (the bundle includes `analysis/contamination_summary.csv` for replication). Our run demonstrates that following these steps makes both positive and null outcomes auditable and actionable.

Concluding interpretive note. The experiment as executed is scientifically honest, reproducible, and valuable: it demonstrates a fully-articulated pipeline and identifies a methodological pitfall that can silently convert an intended independent test into a deterministic sameness check. Because all artifacts and SHA-256 fingerprints (including the immutable initial master prompt `provenance/master_prompt.txt` SHA-256 `1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15` and the preregistration SHA-256 `7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdded71a3`) are published, teams can (and should) deterministically transform this reproducible baseline into an informative assessment of independent generative reliability via the minimal, artifact-grounded reruns described above.

VI. LIMITATIONS

- Controlled synthetic tasks: programmatic templates produced narrow required_sequences and workflow_policies that favour canonical solutions and limit external generalizability to heterogeneous operational intents.
- Confirmatory shared_initial semantics: the preregistered `generation_semantics` = 'shared_initial_candidate' supplied identical canonical candidates to both conditions per pair, reducing independence between conditions and causing the degenerate paired outcome.
- Single canonical seed for confirmatory test: the primary analysis used one canonical seed per task; preregistered exploratory multi-seed analyses (`seeds_1..4`) were not included in the confirmatory inference reported here.
- Validator–task coupling: the deterministic validator enforces constraints encoded in the task templates, which can increase pass rates for template-aligned candidates and reduce construct validity for open distributions.
- Possible training-data exposure: preregistered contamination heuristics found no flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining.
- Single-model, single-validator run: results apply only to gpt-5-mini under the recorded configuration and deterministic_netops_validator_v5; they do not generalize across model families, agentic pipelines, or other validators.

VII. CONCLUSION

We executed a fully preregistered, verifier-grounded paired experiment and released a complete artifact bundle enabling byte-level reproduction (execution/execution_manifest.json and analysis/*). Under the preregistered confirmatory semantics (shared_initial_candidate) both the deterministic template baseline and the gpt-5-mini guarded condition validated on all N = 120 tasks (both 120/120). Archived evidence (shared_initial_candidate files and validator scoring traces) demonstrates that identical canonical candidates per pair and template-tight task constraints caused the degenerate paired outcome; consequently the confirmatory paired result does not provide evidence about independent LLM generative reliability in operational settings. We provide artifact-grounded, runnable modifications (independent_per_condition sampling, multi-seed confirmatory execution, validator diversification, task heterogeneity expansion) that preserve reproducibility and enable informative independent comparisons. All execution and analysis artifacts—including the immutable master prompt reference (provenance/master_prompt.txt; SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba) and the preregistration SHA-256—are included in the submission bundle to permit exact audit and follow-up experiments. Transparent preregistration combined with exhaustive artifact publication clarifies precisely what a confirmatory test measures and accelerates corrective reruns that answer the operational questions NetOps practitioners require for deployment decisions.

DISCLOSURE STATEMENT

Mandatory disclosure (CNSM Agentic AI Researcher track). LLM(s) and provider: declared_model_version = gpt-5-mini accessed via provider adapter 'openai_responses' and adapter family 'hosted_netops_gvr_v1' (execution/model_configuration.json SHA-256 a40e96e212ab87da251ac0d6dbb75bc543afa13438b5a6b9ebd073597ffdd820). Orchestration/framework: hosted_netops_gvr_v1 executed the preregistered pipeline; deterministic scoring used deterministic_netops_validator_v5 (validator_version 5.0). Preregistration: CAND-2026-001-NetConfig-Semantic-v1 (frozen prior to execution); preregistration SHA-256 7db1663cf3982c8ea1e16c3dd7c0348f356bb4cbe3978ecc75bb59fdedb71a37. Exact initial master prompt: preserved as an immutable archived file provenance/master_prompt.txt (SHA-256 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the initial master prompt before the autonomy lock. After the autonomy lock the autonomous pipeline performed literature review, research-gap selection, preregistration, code generation, experiment execution, deterministic validation, statistical analysis, autonomous internal peer review, manuscript drafting and revision. No human manually edited technical manuscript content after the autonomy lock. Preregistered confirmatory

generation semantics and consequence: confirmatory generation_semantics was set to 'shared_initial_candidate' so each pair received an identical canonical candidate for both baseline and guarded episodes; representative shared candidate artifacts: execution/responses/task-000001-shared-initial.txt (SHA-256 8f38c8becd7e1e36...), execution/responses/task-000002-shared-initial.txt (SHA-256 ae967f70dfb6c...), execution/responses/task-000003-shared-initial.txt (SHA-256 f7f4db249ecbbce...). Validator and scoring: deterministic_netops_validator_v5 performed normalized parsing, intent constraint checking, transient safety checks, and emitted per-episode JSON scoring traces archived under execution/scoring/*json (representative SHA-256s: execution/scoring/task-000001-guarded.json SHA-256 0d56c1add7c5a57f...). Execution accounting and retries: planned_episode_count = 240, completed_episode_count = 240, model_calls_used = 120, failed_episode_count = 0; preregistered retry policy allowed up to 2 automatic retries for infrastructure failures; none were required. Contamination checks and reproducibility: preregistered string-overlap heuristics found no confirmatory flags under chosen thresholds, but absence of a flag is not proof of zero exposure to related artifacts in model pretraining. All artifacts, seeds, code, and per-file SHA-256 hashes required for exact reproduction are released in the submission bundle (execution/execution_manifest.json contains the exhaustive manifest). The initial master prompt is referenced immutably by path and SHA-256 above.

REFERENCES

- [1] <https://doi.org/10.1145/3603269.3604866>
- [2] <https://doi.org/10.23919/cnsm62983.2024.10814407>
- [3] <https://doi.org/10.1002/nem.70029>